

ADR-ECPPP — Guía de Defensa del Proyecto

Plataforma Académica ECPMP · Avance de Proyecto · 2026

Índice

- 1. [Resumen Ejecutivo](#)
- 2. [Arquitectura General](#)
- 3. [Patrones Arquitectónicos y de Diseño](#)
- 4. [Principios SOLID](#)
- 5. [Stack Tecnológico y Librerías](#)
- 6. [Base de Datos](#)
- 7. [Seguridad](#)
- 8. [Almacenamiento de Archivos](#)
- 9. [Despliegue](#)
- 10. [Calidad de Software — ISO 25010](#)
- 11. [CI/CD y Testing](#)
- 12. [Módulos del Sistema](#)
- 13. [Banco de Preguntas y Respuestas](#)

1. Resumen Ejecutivo

ECPMP (Plataforma Académica) es un sistema web de gestión educativa institucional. Su objetivo es centralizar en una única plataforma el control de usuarios (estudiantes, docentes, inspectores y secretaría), la estructura académica (periodos, asignaturas, paralelos, matrículas), las calificaciones, la asistencia, las solicitudes estudiantiles y las notificaciones internas.

Atributo	Valor
Lenguaje	Python 3.12

Atributo	Valor
Framework Backend	Django 5.1.7
Base de datos	PostgreSQL 18
Frontend	Django Templates + Tailwind CSS + Alpine.js
Arquitectura	Hexagonal / DDD por Bounded Context
Cobertura de tests	≥ 70% (umbral forzado en CI)
Control de calidad	ISO/IEC 25010
CI/CD	GitHub Actions

2. Arquitectura General

2.1 ¿Por qué se eligió Arquitectura Hexagonal + DDD?

Antes de describir la arquitectura, es importante justificar **por qué** se tomó esta decisión frente a la alternativa más sencilla (Django puro en capas MVC/MVT convencionales).

El problema con un Django "clásico" es que la lógica de negocio termina mezclada con el framework: las validaciones viven en los modelos ORM, las reglas de negocio aparecen en las vistas, y cambiar cualquier pieza requiere entender cómo Django internamente maneja sus dependencias. Esto funciona bien para proyectos pequeños, pero genera deuda técnica acumulada en sistemas que crecen.

La **Arquitectura Hexagonal** resuelve esto con un principio central: **el núcleo de la aplicación (la lógica de negocio) no debe saber nada del mundo exterior** — ni de la base de datos, ni del framework web, ni del proveedor de email. Todo lo que es "infraestructura" queda afuera del núcleo y se conecta mediante interfaces.

Complementariamente, **Domain-Driven Design (DDD)** provee el vocabulario y las herramientas para modelar ese núcleo correctamente: entidades, objetos de valor, agregados, repositorios.

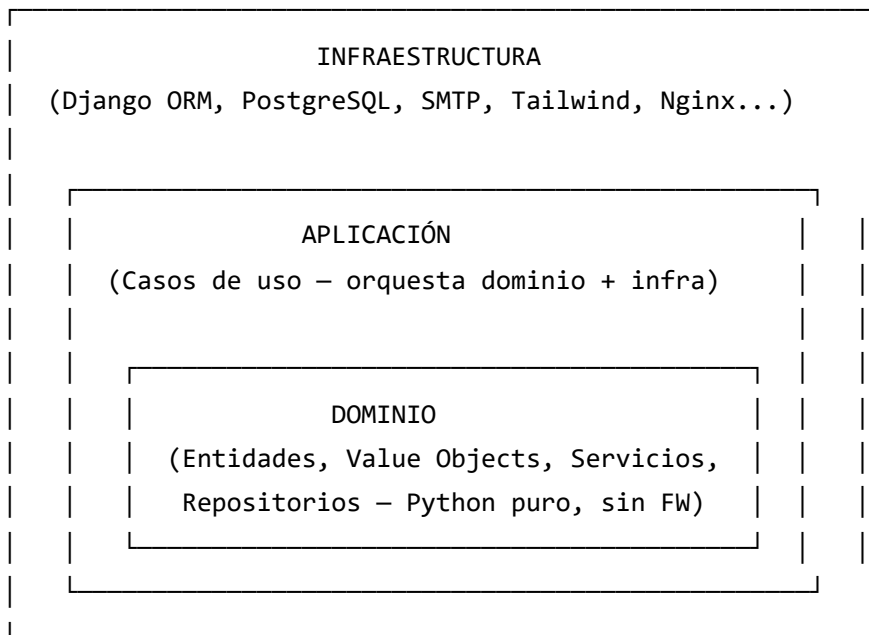
Justificación técnica para ECPPP:

- El sistema modela un dominio complejo con reglas de negocio no triviales (cupos, conflictos de horario, estados de solicitudes, periodos académicos con restricciones de duración).

- Esas reglas deben ser verificables con tests que no necesiten levantar Django ni conectarse a una BD.
- El sistema debe poder migrar de motor de base de datos, proveedor de email o incluso framework sin reescribir la lógica de negocio.

2.2 Arquitectura Hexagonal — Concepto y Aplicación

La Arquitectura Hexagonal (también llamada "Ports & Adapters", propuesta por Alistair Cockburn) divide la aplicación en tres zonas concéntricas:



La regla de dependencia es unidireccional: las capas externas conocen las internas, pero nunca al revés.

Presentación → Aplicación → Dominio ← Infraestructura

Esto significa:

- El **dominio** no importa Django. Ni siquiera sabe que existe una base de datos.
- La **aplicación** orquesta el dominio usando sus interfaces (repositorios abstractos).
- La **infraestructura** implementa esas interfaces con Django ORM, SMTP, etc.
- La **presentación** (vistas Django) llama a la aplicación y devuelve respuestas HTTP.

¿Qué son los "Puertos"? Son las interfaces abstractas que el dominio define. Ejemplo: `IUsuarioRepository` define `guardar(usuario)` , `buscar_por_email(email)` , etc.

¿Qué son los "Adaptadores"? Son las implementaciones concretas de esos puertos. Ejemplo: `DjangoUsuarioRepository` implementa `IUsuarioRepository` usando Django ORM.

2.3 Domain-Driven Design — Conceptos Aplicados

Bounded Context (Contexto Delimitado)

Un Bounded Context es un límite explícito dentro del cual un modelo de dominio específico tiene coherencia y consistencia. En ECPPP, cada app Django es un Bounded Context:

Bounded Context	Responsabilidad
usuarios	Identidad, autenticación, auditoría, OTP
academico	Periodos, tipos de licencia, asignaturas, paralelos, matrículas
calificaciones	Evaluaciones y notas
asistencia	Registro de asistencia diaria
solicitudes	Peticiones estudiantiles de rectificación y justificación
secretaria	Funciones administrativas transversales
notificaciones	Sistema de alertas internas

Cada contexto tiene su propio lenguaje ubícuo (ubiquitous language): las palabras "Paralelo", "Matrícula", "Tipo de Licencia" tienen significado preciso dentro de `academico` y no se mezclan con los términos de otros contextos.

Entidad de Dominio (Entity)

Una **entidad** es un objeto que tiene identidad propia a lo largo del tiempo. En ECPPP, `Usuario` es una entidad: aunque cambie su nombre, su email o su rol, sigue siendo el mismo usuario porque tiene un `id` único.

Las entidades de dominio en ECPPP son `@dataclass(frozen=True)` — objetos inmutables que representan el estado del dominio sin depender de Django:

```
# apps/usuarios/domain/entities.py – Python puro, cero Django
@dataclass(frozen=True)
class UsuarioEntity:
    username: str
    email: str
    rol: Rol
    cedula: str          # obligatorio, inmutable post-creación
    first_name: str = ""
    is_active: bool = True
```

¿Por qué frozen=True ? Porque las entidades de dominio no deben ser mutadas libremente. Si se necesita "cambiar" una entidad, se crea una nueva con los valores actualizados. Esto previene efectos secundarios inesperados.

Value Object (Objeto de Valor)

Un **Value Object** se define por sus atributos, no por su identidad. Dos Value Objects con los mismos atributos son iguales. No tienen ID.

En ECPPP, `PeriodoEntity`, `AsignaturaEntity`, `CalificacionEntity` son Value Objects: lo que importa son sus datos (nombre, fechas, nota), no si son "el mismo objeto" en memoria.

Aggregate Root

Es la entidad raíz de un conjunto de objetos relacionados que se tratan como una unidad. Toda modificación del agregado pasa por su raíz.

En ECPPP:

- `Paralelo` es el Aggregate Root del conjunto $\{\text{Paralelo} \rightarrow \text{Matrícula} \rightarrow \text{BloqueHorario}\}$
- `Evaluacion` es el Aggregate Root del conjunto $\{\text{Evaluacion} \rightarrow \text{Calificacion}\}$

Esto garantiza que las invariantes del dominio (ej.: cupo máximo de un paralelo) se validan en un solo lugar.

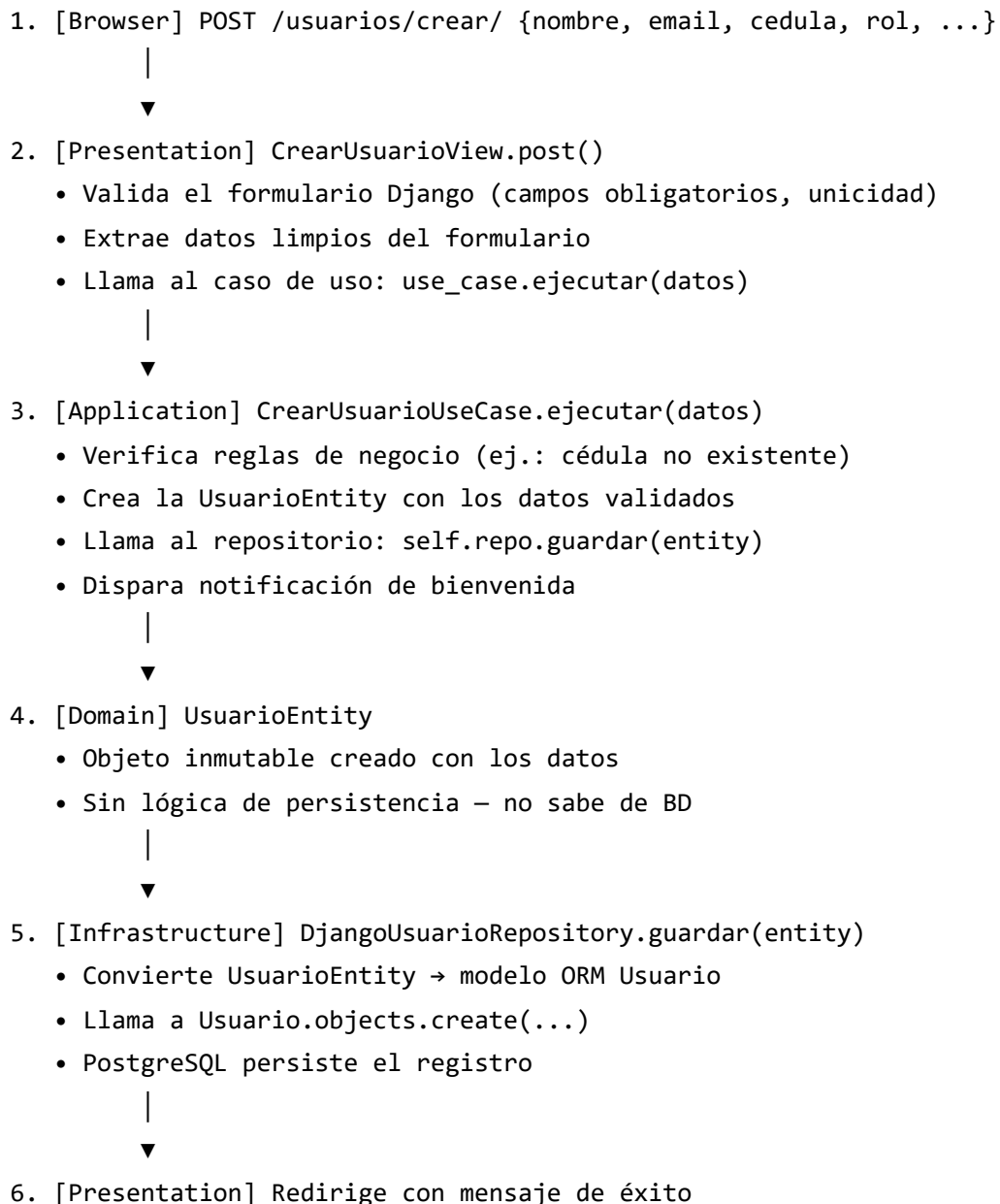
2.4 Estructura de Capas por App

```
apps/<contexto>/
├── domain/
│   ├── entities.py      ← Entidades y Value Objects (Python puro)
│   ├── value_objects.py ← Value Objects adicionales
│   ├── repositories.py  ← Interfaces abstractas (Puertos)
│   ├── services.py      ← Lógica de negocio pura
│   └── exceptions.py    ← Excepciones propias del dominio
├── application/
│   └── use_cases.py      ← Casos de uso: orquesta dominio + repositorios
├── infrastructure/
│   ├── models.py        ← Modelos ORM de Django (Adaptador de BD)
│   ├── repositories.py  ← Implementación concreta de los Puertos
│   ├── auth_backend.py  ← Adaptador de autenticación (solo en usuarios)
│   └── email_service.py ← Adaptador de email (solo en usuarios)
└── presentation/
    ├── views.py         ← Vistas Django (controladores HTTP)
    ├── forms.py         ← Formularios Django
    └── urls.py          ← Rutas de la app
```

Regla de oro que se respeta en el código: ningún archivo dentro de `domain/` contiene `import django` ni `from django import ...`. Es Python puro.

2.5 Flujo Completo de una Solicitud HTTP

Para concretar cómo fluye una request real, tomemos el ejemplo de "Secretaría crea un nuevo usuario":



Este flujo garantiza que si mañana se decide cambiar PostgreSQL por otro motor, solo se reemplaza el paso 5.

3. Patrones Arquitectónicos y de Diseño

3.1 Patrón: Repository (Repositorio)

¿Qué es?

El Repository Pattern abstrae el acceso a los datos detrás de una interfaz. La capa de dominio define

qué operaciones necesita (guardar, buscar, eliminar), y la capa de infraestructura define *cómo* se hacen realmente con la tecnología elegida.

¿Por qué se necesita?

Sin este patrón, si la lógica de negocio llama directamente a `Usuario.objects.filter(...)`, está acoplada a Django ORM. Si se cambia de Django a FastAPI, o de PostgreSQL a MongoDB, hay que reescribir toda la lógica de negocio. Con el Repository Pattern, el dominio solo habla con la interfaz abstracta.

¿Cómo se aplica en ECPMP?

```
# domain/repositories.py – LA INTERFAZ (el "puerto")
# No importa Django. Es un contrato abstracto.
from abc import ABC, abstractmethod

class IUserRepository(ABC):
    @abstractmethod
    def guardar(self, usuario: UsuarioEntity) -> UsuarioEntity: ...

    @abstractmethod
    def buscar_por_email(self, email: str) -> Optional[UsuarioEntity]: ...

    @abstractmethod
    def buscar_por_cedula(self, cedula: str) -> Optional[UsuarioEntity]: ...

# infrastructure/repositories.py – LA IMPLEMENTACIÓN (el "adaptador")
# Aquí sí importa Django ORM.
class DjangoUsuarioRepository(IUsuarioRepository):
    def buscar_por_email(self, email: str) -> Optional[UsuarioEntity]:
        try:
            orm_user = Usuario.objects.get(email=email)
            return self._to_entity(orm_user)
        except Usuario.DoesNotExist:
            return None
```

Beneficio concreto: Si se decidiera agregar caché Redis para las búsquedas de usuario, solo se crea `CachedUsuarioRepository` que delega en `DjangoUsuarioRepository`. La lógica de negocio no sabe nada del cambio.

3.2 Patrón: Strategy (Estrategia)

¿Qué es?

Define una familia de algoritmos intercambiables, los encapsula y los hace intercambiables en tiempo de ejecución. El cliente usa la interfaz sin saber qué algoritmo concreto está ejecutando.

¿Por qué se necesita?

En el contexto de autenticación, diferentes sistemas (o el mismo en diferentes contextos) pueden necesitar distintas formas de verificar identidad. Django diseñó su sistema de autenticación usando exactamente este patrón.

¿Cómo se aplica en ECPDP?

Django permite definir múltiples "backends" de autenticación en `AUTHENTICATION_BACKENDS`. Cada backend es una estrategia concreta:

```
# config/settings/base.py
AUTHENTICATION_BACKENDS = [
    "apps.usuarios.infrastructure.auth_backend.ECPDPAuthBackend", # Estrategia 1
    "django.contrib.auth.backends.ModelBackend",                # Estrategia 2
]
```

`ECPDPAuthBackend` es la estrategia primaria: autentica por **email + contraseña + rol**. Si falla, Django prueba con `ModelBackend` (estrategia de fallback): autentica por **username + contraseña** para que el Django Admin siga funcionando.

```
# infrastructure/auth_backend.py
class ECPDPAuthBackend(ModelBackend):
    def authenticate(self, request, email=None, password=None, tipo_usuario=None, **kwargs):
        # Estrategia específica de ECPDP: triple verificación
        user = Usuario.objects.get(email=email)
        if not user.check_password(password): return None
        if tipo_usuario and user.rol != tipo_usuario: return None
        return user
```

Beneficio concreto: Si en el futuro se quiere agregar autenticación con Google OAuth, se agrega una tercera estrategia sin tocar las dos existentes.

3.3 Patrón: Template Method (Método Plantilla)

¿Qué es?

Define el esqueleto de un algoritmo en una clase base, dejando que las subclasses sobrescriban pasos específicos sin cambiar la estructura general del algoritmo.

¿Por qué se necesita?

Permite que Django tenga un sistema de usuarios robusto y estandarizado, mientras cada proyecto puede personalizarlo agregando campos o comportamientos propios.

¿Cómo se aplica en ECPPP?

Django define `AbstractUser` con el algoritmo completo de un usuario: autenticación, permisos, hashing de contraseñas, sesiones. ECPPP extiende esa base sobrescribiendo (agregando) los campos propios:

```
# infrastructure/models.py
class Usuario(AbstractUser): # Template Method: AbstractUser define el "esqueleto"
    # ECPPP "rellena" los pasos adicionales que le corresponden al dominio académico
    rol = models.CharField(max_length=20, choices=Rol.choices)
    cedula = models.CharField(max_length=13, unique=True)
    intentos_fallidos = models.PositiveIntegerField(default=0)
    bloqueado_hasta = models.DateTimeField(null=True, blank=True)
    debe_cambiar_password = models.BooleanField(default=False)
```

Beneficio concreto: Todo el sistema de hashing, sesiones, permisos y admin de Django funciona automáticamente. Solo se "rellenan" los campos que hacen único a ECPPP. Sin este patrón, habría que reimplementar todo el sistema de autenticación desde cero.

3.4 Patrón: Decorator (Decorador)

¿Qué es?

Permite agregar responsabilidades a un objeto (o función) dinámicamente, sin modificarlo. Se "envuelve" el objeto original con un wrapper que agrega comportamiento antes o después.

¿Por qué se necesita?

Las vistas Django necesitan protecciones transversales: verificar que el usuario está autenticado, verificar que tiene el rol correcto. Repetir esa lógica en cada vista sería duplicación masiva.

¿Cómo se aplica en ECPMP?

```
# presentation/views.py
@login_required                                # Decorador 1: verifica autenticación
@permission_required('usuarios.add_usuario')  # Decorador 2: verifica permiso
def crear_usuario(request):
    # La lógica de la vista solo corre si AMBOS decoradores pasan
    ...
```

El decorador `@login_required` "envuelve" la función `crear_usuario`: antes de ejecutarla, verifica si `request.user.is_authenticated`. Si no, redirige al login. La vista en sí no sabe nada de esta verificación.

Middleware como Decorator: El `ForzarCambioPasswordMiddleware` es un decorador a nivel de toda la aplicación — intercepta cada request antes de que llegue a cualquier vista:

Request HTTP → [ForzarCambioPassword] → [CSRF] → [Auth] → Vista

Si `debe_cambiar_password = True`, el middleware redirige sin dejar pasar la request a la vista destino.

Beneficio concreto: Las reglas de acceso se definen en un solo lugar. Cambiar la política de autorización no requiere tocar las vistas.

3.5 Patrón: Observer / Signals

¿Qué es?

Define una dependencia uno-a-muchos entre objetos: cuando un objeto cambia de estado, todos sus observadores son notificados y actualizados automáticamente. El emisor no conoce a sus observadores.

¿Por qué se necesita?

Cuando ocurre un evento de negocio (usuario creado, solicitud aprobada), varios módulos necesitan reaccionar: enviar email, crear notificación, registrar en auditoría. Sin Observer, el caso de uso tendría que llamar explícitamente a cada módulo, creando acoplamiento fuerte.

¿Cómo se aplica en ECPMP?

Django implementa este patrón con su sistema de `signals`:

```
# Emisor (el evento ocurre aquí — no sabe quién escucha)
from django.db.models.signals import post_save
post_save.send(sender=Usuario, instance=usuario, created=True)

# Observador (reacciona al evento — no sabe quién emitió)
@receiver(post_save, sender=Usuario)
def enviar_bienvenida(sender, instance, created, **kwargs):
    if created:
        email_service.enviar_bienvenida(instance.email)
```

El módulo de notificaciones puede "escuchar" eventos del módulo académico sin que el módulo académico importe nada de notificaciones.

Beneficio concreto: El desacoplamiento es real — se puede agregar un nuevo observador (ej.: enviar SMS) sin modificar el código que emite el evento.

3.6 Patrón: Factory (Fábrica) — en Testing

¿Qué es?

Centraliza la creación de objetos complejos. En vez de instanciar objetos directamente en los tests (con todos sus campos requeridos), una Factory sabe cómo crear objetos válidos con datos razonables por defecto.

¿Por qué se necesita?

Los tests necesitan datos de prueba. Si cada test crea sus objetos manualmente, cualquier cambio en el modelo rompe docenas de tests. Además, la creación de fixtures repetitiva infla el código de test y lo hace ilegible.

¿Cómo se aplica en ECPMP?

Con `factory-boy` :

```
# tests/factories.py
class UsuarioFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = Usuario
    username = factory.Sequence(lambda n: f"user{n}")
    email = factory.LazyAttribute(lambda o: f"{o.username}@test.com")
    cedula = factory.Sequence(lambda n: f"17{n:08d}")
    rol = "estudiante"
    password = factory.PostGenerationMethodCall("set_password", "Test@1234")
```

En los tests:

```
def test_bloqueo_cuenta():
    user = UsuarioFactory()          # Objeto válido en una línea
    for _ in range(5):
        login_fallido(user.email)
    assert user.bloqueado_hasta is not None
```

Beneficio concreto: Los tests son concisos y resistentes a cambios del modelo. Si se agrega un campo obligatorio nuevo a `Usuario`, solo se actualiza `UsuarioFactory`.

3.7 Patrón: Chain of Responsibility (vía Middleware)

¿Qué es?

Pasa una solicitud a través de una cadena de manejadores. Cada manejador decide si procesa la solicitud o la pasa al siguiente. Ningún manejador necesita saber quién es el siguiente.

¿Por qué se necesita?

Las preocupaciones transversales (seguridad, sesiones, logging, CSRF) no deben vivir en cada vista. Necesitan ejecutarse sistemáticamente para toda solicitud.

¿Cómo se aplica en ECPMP?

El stack de middlewares de Django es exactamente esta cadena:

```
# config/settings/base.py
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",          # Manejador 1
    "django.contrib.sessions.middleware.SessionMiddleware",  # Manejador 2
    "django.middleware.common.CommonMiddleware",             # Manejador 3
    "django.middleware.csrf.CsrfViewMiddleware",             # Manejador 4
    "django.contrib.auth.middleware.AuthenticationMiddleware", # Manejador 5
    "apps.usuarios.presentation.middleware.ForzarCambioPasswordMiddleware", # Manejador 6 (custom)
    "django.contrib.messages.middleware.MessageMiddleware",   # Manejador 7
    "django.middleware.clickjacking.XFrameOptionsMiddleware", # Manejador 8
]
```

Cada request pasa por los 8 manejadores en orden. Si alguno rechaza la request (ej.: CSRF inválido), la cadena se interrumpe y la vista nunca llega a ejecutarse.

Beneficio concreto: `ForzarCambioPasswordMiddleware` se insertó en posición 6 sin tocar ninguna otra pieza. Se puede quitar o reordenar en una línea.

3.8 Patrón: MVT (Model-View-Template)

¿Qué es?

Es la variante de Django del patrón MVC (Model-View-Controller). El browser hace un request → la URL lo enruta al View → el View interactúa con el Model → el Template renderiza el HTML → Django devuelve la respuesta.

Roles en Django:

- **Model:** Lógica de datos y acceso a la BD (en ECPMP: la capa de infraestructura + dominio).
- **View:** Recibe el request, orquesta la lógica y decide qué Template usar (en ECPMP: `presentation/views.py`).
- **Template:** HTML dinámico que recibe el contexto del View y lo renderiza (en ECPMP: `templates/`).

¿Por qué se necesita?

Separa las responsabilidades de presentación, lógica y datos. El template no toca la BD; el view no genera HTML; el modelo no sabe del HTTP.

¿Cómo se aplica en ECPMP?

```
URL: /usuarios/crear/
|
▼
View: CrearUsuarioView → Llama al caso de uso
|
▼
Template: usuarios/crear.html → Recibe {form, titulo, errores}
|
▼
HTML renderizado → Response HTTP 200
```

El template solo renderiza lo que el View le pasa en el contexto. El template nunca hace queries a la BD directamente.

3.9 Patrón: Settings por Entorno (Environment-Specific Configuration)

¿Qué es?

Separa la configuración del sistema según el entorno de ejecución (desarrollo, testing, producción), manteniendo una base común y sobrescribiendo solo lo que difiere.

¿Por qué se necesita?

En desarrollo se quiere `DEBUG=True`, poder conectarse a una BD local y enviar emails a la consola. En producción, `DEBUG=False` es obligatorio (exponer traceback en producción es una vulnerabilidad crítica), se fuerzan cookies HTTPS y se usan credenciales reales.

¿Cómo se aplica en ECPPP?

```
config/settings/
├─ base.py          ← Todo lo compartido (apps, middleware, auth, i18n)
├─ development.py   ← DEBUG=True, BD local, EMAIL_BACKEND=console
└─ production.py    ← DEBUG=False, ALLOWED_HOSTS, cookies HTTPS, headers
```

```
# production.py
from .base import *    # Hereda todo lo de base.py
DEBUG = False
ALLOWED_HOSTS = os.environ.get("DJANGO_ALLOWED_HOSTS", "").split(",")
CSRF_COOKIE_SECURE = True
SESSION_COOKIE_SECURE = True
```

Beneficio concreto: Es imposible que `DEBUG=True` llegue accidentalmente a producción si se usa `production.py`. Las credenciales se leen de variables de entorno, nunca están hardcodeadas.

4. Principios SOLID

Los principios SOLID son 5 principios de diseño orientado a objetos que, aplicados en conjunto, producen sistemas más mantenibles, extensibles y resistentes al cambio.

S — Single Responsibility Principle (Responsabilidad Única)

¿Qué dice? Cada módulo o clase debe tener una única razón para cambiar.

En ECPPP:

Capa	Su única responsabilidad
domain/entities.py	Representar el estado del dominio. Cambia solo si cambia el negocio.
infrastructure/models.py	Mapear entidades a la BD. Cambia solo si cambia el esquema.
application/use_cases.py	Orquestar el flujo de un caso de uso. Cambia solo si cambia el proceso.
presentation/views.py	Manejar HTTP: recibir request, devolver response. Cambia solo si cambia la UI.

Ejemplo concreto: `RegistroAuditoria` tiene solo una responsabilidad: registrar eventos de seguridad. No procesa lógica de negocio, no envía emails, no calcula nada. Si cambio el formato del log, solo toco ese modelo.

O — Open/Closed Principle (Abierto/Cerrado)

¿Qué dice? Las entidades de software deben estar abiertas para extensión, pero cerradas para modificación.

En ECPPP:

Los repositorios del dominio son interfaces (ABC). Si mañana se quiere agregar caché Redis para las búsquedas de usuario, se **extiende** (se crea `CachedUsuarioRepository`) sin **modificar** `DjangoUsuarioRepository` ni el caso de uso.

```
# EXTENSIÓN sin modificación
class CachedUsuarioRepository(IUsuarioRepository):
    def __init__(self, base_repo, cache):
        self.base = base_repo
        self.cache = cache

    def buscar_por_email(self, email):
        cached = self.cache.get(f"user:{email}")
        if cached: return cached
        result = self.base.buscar_por_email(email)
        self.cache.set(f"user:{email}", result, ttl=300)
        return result
```

El caso de uso no cambia. La vista no cambia. Solo se swapea el repositorio en la inyección de dependencias.

L — Liskov Substitution Principle (Sustitución de Liskov)

¿Qué dice? Los objetos de una subclase deben poder sustituir a los de su clase base sin alterar el comportamiento del programa.

En ECPPP:

`ECPPPAuthBackend` extiende `ModelBackend` . En cualquier lugar donde Django espera un backend de autenticación, `ECPPPAuthBackend` puede usarse sin romper nada. Hereda `user_can_authenticate()` correctamente y solo sobrescribe `authenticate()` con reglas adicionales, no contradictorias.

Si se creara un `LDAPAuthBackend` para autenticar contra un servidor corporativo, también podría sustituir a `ModelBackend` en la lista de backends sin que Django se entere de la diferencia.

I — Interface Segregation Principle (Segregación de Interfaces)

¿Qué dice? Ningún cliente debería verse forzado a depender de métodos que no usa.

En ECPPP:

Cada repositorio de dominio expone solo las operaciones que su contexto necesita. El repositorio de `academico` no tiene métodos de `usuarios` y viceversa. No hay un `IGodRepository` con todos los métodos mezclados.

```
# IParaleloRepository – solo lo que necesita el contexto académico
class IParaleloRepository(ABC):
    @abstractmethod
    def guardar(self, paralelo: ParaleloEntity): ...

    @abstractmethod
    def obtener_por_id(self, id: int): ...

    @abstractmethod
    def verificar_conflicto_horario(self, docente_id, bloques): ...
```

Si el contexto de calificaciones necesita acceder a paralelos, usa `IParaleloRepository` (una interfaz pequeña y específica), no un repositorio genérico gigante.

D — Dependency Inversion Principle (Inversión de Dependencias)

¿Qué dice? Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones.

En ECPPP:

El caso de uso (alto nivel) no importa `DjangoUsuarioRepository` (bajo nivel). Depende de `IUsuarioRepository` (abstracción):

```
# application/use_cases.py
class CrearUsuarioUseCase:
    def __init__(self, repo: IUserarioRepository): # Depende de la interfaz
        self.repo = repo

    def ejecutar(self, datos):
        entity = UsuarioEntity(**datos)
        return self.repo.guardar(entity)
```

La implementación concreta se inyecta desde afuera (en la vista o en la configuración de la app). El dominio y la aplicación nunca "miran hacia abajo" en la jerarquía de dependencias.

5. Stack Tecnológico y Librerías

5.1 Backend

Librería	Versión	Propósito	Por qué se eligió
Django	5.1.7	Framework web principal	ORM, migraciones, auth, admin, CSRF, sesiones incluidos. Madura (15+ años), gran ecosistema, convenciones claras.
Django REST Framework	3.16.0	API REST	Estándar de facto para APIs con Django. Serializers, ViewSets, paginación integrada. Usado en módulo académico.
psycopg2-binary	2.9.10	Adaptador PostgreSQL	Es el adaptador oficial y recomendado de Django para PostgreSQL. La variante <code>binary</code> incluye las librerías compiladas sin dependencias de sistema.
python-dotenv	1.1.0	Variables de entorno	Carga el <code>.env</code> automáticamente. Simple, sin dependencias, estándar de la industria para la configuración 12-factor.
python-dateutil	2.9.0	Manejo de fechas	Operaciones avanzadas de fechas necesarias para la validación de duración de periodos académicos (diferencias en meses).

5.2 Frontend

Librería	Versión	Propósito	Por qué se eligió
Tailwind CSS	3.4.17	Framework CSS	Sistema de diseño basado en utilidades. Sin CSS sobrante (purging), tokens configurables, sin conflictos con componentes de Django. Diseño consistente a bajo costo.
Alpine.js	3.15.9	Reactividad frontend	Reactividad declarativa en el HTML sin necesidad de un SPA completo. Footprint mínimo (15KB), sin build step, perfectamente compatible con SSR de Django.
Tom Select	2.3.1	Selects enriquecidos	Selects con búsqueda, accesibles, personalizables. Necesario para listas largas (docentes, asignaturas, estudiantes) donde el <code><select></code> nativo no es suficiente.
Inter (Google Fonts)	—	Tipografía	Fuente sans-serif moderna, altamente legible en pantalla, variable font con múltiples pesos para jerarquía visual.

Nota sobre CDN vs. self-hosted: Actualmente los assets frontend se cargan desde CDN (jsDelivr, Google Fonts). En producción se evaluará self-hosting para: (1) eliminar dependencias externas, (2) mejorar tiempo de carga con assets servidos desde el mismo VPS, y (3) funcionar sin acceso a internet externo.

5.3 Calidad, Testing y Linting

Herramienta	Versión	Propósito	Por qué se eligió
pytest	8.3.5	Framework de tests	Más expresivo y extensible que el <code>unittest</code> estándar. Fixtures potentes, plugins, mejor output de errores. Estándar moderno de Python.
pytest-django	4.10.0	Integración pytest+Django	Provee fixtures de Django (<code>db</code> , <code>client</code> , <code>rf</code>) para pytest. Maneja la configuración del settings de test y el ciclo de vida de la BD de test.

Herramienta	Versión	Propósito	Por qué se eligió
coverage	7.8.0	Cobertura de código	Mide qué porcentaje del código es ejercitado por los tests. Integrado en CI para forzar umbral mínimo.
factory-boy	3.3.1	Generación de fixtures	Crea objetos de test válidos con datos generados. Evita duplicación masiva en setup de tests. Compatible con Django ORM.
flake8	7.2.0	Linter PEP8	Verifica conformidad con la guía de estilo oficial de Python. Detecta errores sintácticos y malas prácticas antes de runtime.
black	25.1.0	Formateador automático	Formateador "opinionado": hay una sola forma de formatear el código. Elimina debates sobre estilo y garantiza uniformidad total.

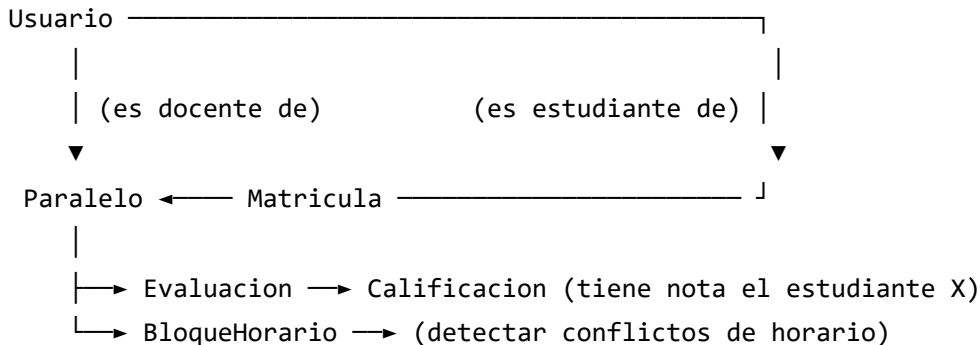
6. Base de Datos

6.1 Motor elegido: PostgreSQL (SQL Relacional)

La decisión de usar PostgreSQL sobre SQLite (default de Django) o MongoDB (NoSQL) fue deliberada y técnicamente justificada.

¿Por qué SQL y no NoSQL?

La pregunta fundamental es: ¿qué tipo de datos maneja el sistema? En ECPPP, los datos son **altamente relacionales y estructurados**:



Cada entidad referencia a otras mediante Foreign Keys. Las consultas más frecuentes son joins: "dame todas las calificaciones del estudiante X en el periodo Y". PostgreSQL maneja esto de forma nativa, eficiente e íntegra.

Un motor documental como MongoDB almacenaría cada documento de forma denormalizada. Para este dominio, eso significaría duplicar datos (el nombre del docente en cada paralelo, en vez de una FK) y perder la garantía de integridad referencial que PostgreSQL garantiza a nivel de motor.

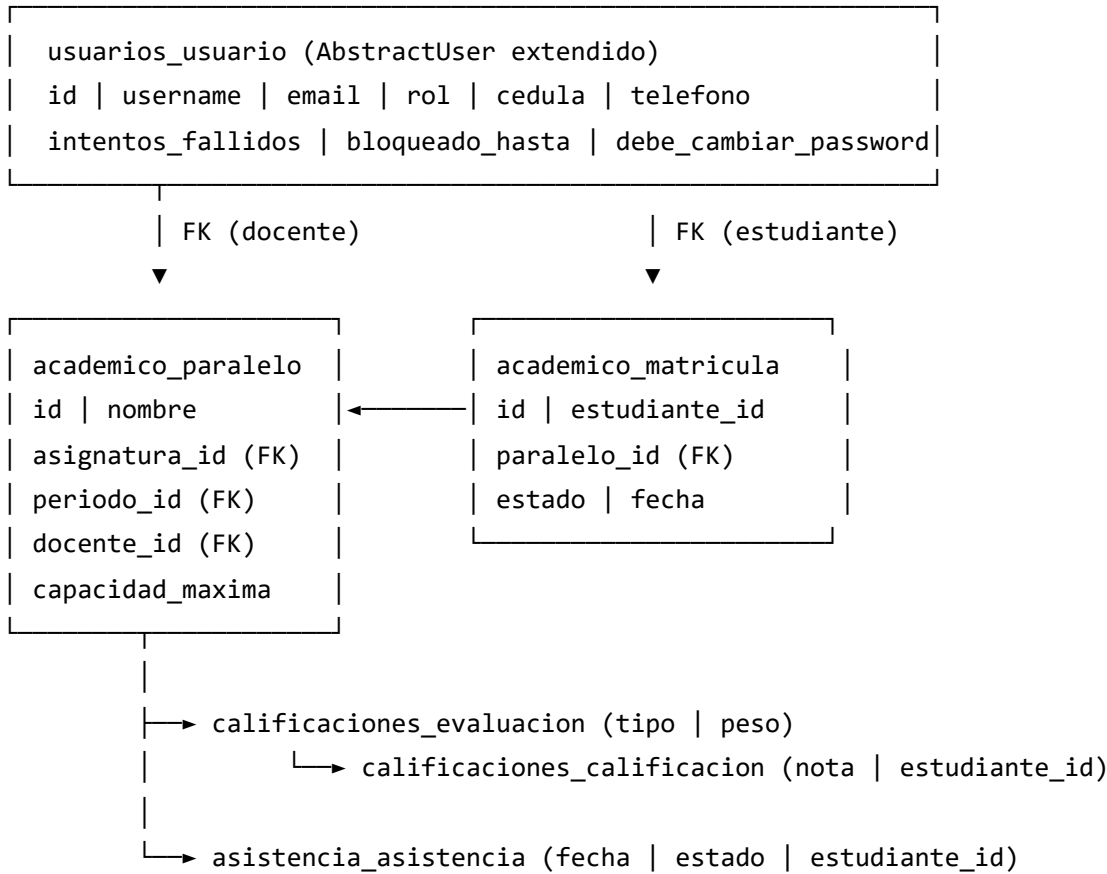
¿Por qué PostgreSQL y no MySQL/MariaDB?

Criterio	PostgreSQL	MySQL
Soporte nativo en Django	Excelente (primer ciudadano)	Bueno
DISTINCT ON , window functions	Sí	Limitado
Tipos de dato avanzados (JSON, Array)	Nativo	Básico
ACID estricto por default	Sí	Depende del engine
Licencia	Open source (libre)	Open source (Oracle)

Propiedades ACID garantizadas:

- **Atomicidad:** Una transacción se completa completa o no se ejecuta. Si al crear un usuario con matrícula falla la matrícula, el usuario tampoco queda creado.
- **Consistencia:** La BD pasa de un estado válido a otro. Las restricciones (unique, FK, check) se validan siempre.
- **Aislamiento:** Las transacciones concurrentes no se interfieren. Dos secretarias creando usuarios simultáneamente no generan cédulas duplicadas.
- **Durabilidad:** Una vez confirmada, la transacción persiste aunque el servidor se apague.

6.2 Diseño del Esquema — Modelos Principales



6.3 Índices de Base de Datos

Los índices aceleran las búsquedas a cambio de un pequeño costo en escrituras. Se definieron índices compuestos en los campos más consultados:

```
# OTPToken: búsquedas frecuentes por usuario + estado + expiración
class Meta:
    indexes = [
        models.Index(fields=["usuario", "usado", "expira_en"])
    ]

# Permite: "dame el token vigente de este usuario" en O(log n)

# RegistroAuditoria: búsquedas por usuario + tipo de acción + tiempo
class Meta:
    indexes = [
        models.Index(fields=["usuario", "accion", "timestamp"])
    ]

# Permite: "dame los últimos 5 logins del usuario X" sin full scan
```

6.4 Migraciones y Versionado del Esquema

Django genera archivos de migración automáticamente cuando cambia un modelo. Estos archivos son código Python versionado en git, lo que significa:

- El historial del esquema de BD está en el repositorio junto al código.
- Cualquier desarrollador puede recrear exactamente la misma BD corriendo `migrate`.
- El CI/CD verifica que las migraciones son consistentes con el código en cada push.
- Las migraciones son reversibles: `migrate academico 0003` vuelve el esquema al estado de esa migración.

7. Seguridad

7.1 Autenticación Reforzada

El sistema implementa un **backend de autenticación personalizado** que exige tres datos para autenticar:

Factor	Campo	Por qué importa
1	email	Identificador único del usuario
2	password	Secreto del usuario
3	rol	Confirma el contexto de acceso

El tercer factor (rol) previene que un usuario con credenciales válidas inicie sesión como un rol que no le corresponde. Ejemplo: un estudiante que conoce las credenciales de un docente no puede autenticarse como docente porque seleccionaría "Estudiante" en el login.

Adicionalmente, para mitigar timing attacks (ataques que miden el tiempo de respuesta para inferir si un usuario existe), el backend ejecuta `set_password()` aunque el usuario no exista:

```
try:
    user = Usuario.objects.get(email=email)
except Usuario.DoesNotExist:
    # Consumir tiempo similar al check_password para no revelar si el usuario existe
    Usuario().set_password(password)
return None
```


7.2 Encriptación de Contraseñas — PBKDF2

Django almacena contraseñas con el formato:

```
algoritmo$iteraciones$salt$hash  
pbkdf2_sha256$870000$randomSalt$base64EncodedHash
```

PBKDF2 (Password-Based Key Derivation Function 2):

1. Toma la contraseña del usuario.
2. Genera un **salt** aleatorio (evita rainbow table attacks — dos usuarios con la misma contraseña tienen hashes distintos).
3. Aplica SHA-256 con el salt **870.000 iteraciones** (Django 5.x). Cada iteración hace el proceso más lento para un atacante que intenta fuerza bruta.
4. El resultado es un hash de 256 bits almacenado en la BD.

Resultado: Si alguien roba la BD, no puede recuperar las contraseñas. Para descifrar una sola contraseña de 8 caracteres con GPU moderna, tardaría meses con PBKDF2 bien configurado.

7.3 Validadores de Contraseña Custom

Además del hash, se valida la fortaleza antes de aceptar una nueva contraseña:

```
AUTH_PASSWORD_VALIDATORS = [  
    {"NAME": "...MinimumLengthValidator",      "OPTIONS": {"min_length": 8}},  
    {"NAME": "...CommonPasswordValidator"},    # 20.000 contraseñas más comunes rechazadas  
    {"NAME": "...NumericPasswordValidator"},   # No puede ser "12345678"  
    {"NAME": "...UppercaseValidator"},         # Al menos una mayúscula (custom)  
    {"NAME": "...SymbolValidator"},            # Al menos un símbolo (custom)  
    {"NAME": "...UserAttributeContainmentValidator"}, # No puede contener nombre/email/rol (cu:  
]
```

El `UserAttributeContainmentValidator` es especialmente sofisticado: normaliza la contraseña y los atributos del usuario quitando tildes (`unicodedata.normalize NFKD`) para que "martinez" no sea válido incluso si el apellido es "Martínez".

7.4 Protección CSRF

CSRF (Cross-Site Request Forgery) es un ataque donde un sitio malicioso engaña al browser del usuario autenticado para hacer requests en su nombre.

Django lo previene con un token secreto único por sesión:

- 1. Al renderizar un formulario, Django incluye `{% csrf_token %}` → un campo hidden con un token.
- 2. Al enviar el formulario, Django verifica que el token del formulario coincide con el de la cookie.
- 3. Un sitio malicioso no puede conocer el token porque está ligado a la sesión del usuario.

```
<!-- Cada formulario en ECPPP tiene esto -->
<form method="POST">
    {% csrf_token %}
    ...
</form>
```

En producción, la cookie CSRF solo se envía por HTTPS (`CSRF_COOKIE_SECURE = True`), eliminando la posibilidad de interceptación en tránsito.

7.5 Seguridad de Sesión

Las sesiones se configuran defensivamente:

Configuración	Valor	Protege contra
<code>SESSION_COOKIE_HTTPONLY = True</code>	JS no puede leer la cookie	XSS que roba la sesión
<code>SESSION_COOKIE_AGE = 3600</code>	Expira en 1 hora	Sesiones robadas que persisten indefinidamente
<code>SESSION_EXPIRE_AT_BROWSER_CLOSE = True</code>	Cierra al cerrar tab	Equipo compartido
<code>SESSION_COOKIE_SECURE = True (prod)</code>	Solo por HTTPS	Interceptación en tránsito

7.6 Cabeceras de Seguridad HTTP

En producción, Django agrega cabeceras de respuesta que instruyen al browser sobre políticas de seguridad:

Cabecera	Valor	Qué hace
X-Content-Type-Options	nosniff	Impide que el browser "adivine" el tipo de contenido (MIME sniffing)

Cabecera	Valor	Qué hace
X-XSS-Protection	1; mode=block	Activa el filtro XSS del browser (legacy, complementario a CSP)
X-Frame-Options	SAMEORIGIN	Solo el mismo sitio puede embeber la app en un iframe (previene clickjacking)

7.7 Auditoría de Seguridad

El modelo `RegistroAuditoria` persiste eventos de seguridad con suficiente contexto para investigar incidentes:

```
class RegistroAuditoria(models.Model):
    usuario      = FK(Usuario, SET_NULL) # SET_NULL: el log persiste si el usuario es eliminado
    accion       = CharField(max_length=50) # "login_exitoso", "login_fallido", "cuenta_bloqueada"
    ip           = GenericIPAddressField() # IP del cliente
    timestamp    = DateTimeField(auto_now_add=True)
    detalle      = TextField() # Contexto adicional
```

El uso de `SET_NULL` (en lugar de `CASCADE`) es crítico: si un usuario es eliminado del sistema, su historial de auditoría se mantiene. Los registros no se borran en cascada porque podrían ser evidencia de acciones inapropiadas.

8. Almacenamiento de Archivos

8.1 Diferencia entre Static Files y Media Files

Es fundamental entender esta distinción en Django:

Tipo	Qué son	Cuándo cambian	Dónde viven
Static Files	CSS, JS, imágenes del sistema, favicon	Con cada deploy del código	<code>static/</code> en el repo → <code>staticfiles/</code> en prod
Media Files	Archivos subidos por usuarios (PDFs, imágenes)	En tiempo de ejecución	<code>media/</code> (fuera del código)

Los static files se "congelan" con `collectstatic` durante el deploy y nunca cambian entre deploys. Los media files son dinámicos — un inspector puede subir una imagen de evidencia a las 2pm y tiene que estar disponible a las 2:01pm.

8.2 Configuración Actual

```
# config/settings/base.py
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"] # Fuente en desarrollo
STATIC_ROOT = BASE_DIR / "staticfiles" # Destino de collectstatic (producción)

MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media" # Carpeta de uploads en el servidor
```

Los archivos subidos se almacenan en el **filesystem local** del servidor. Django los sirve en desarrollo. En producción, Nginx los sirve directamente.

La configuración `X_FRAME_OPTIONS = "SAMEORIGIN"` (en vez del más restrictivo `DENY`) fue una decisión deliberada: el módulo del inspector necesita previsualizar PDFs dentro de `<iframe>` en la misma página. Con `DENY`, los iframes bloqueados romperían esa funcionalidad. Con `SAMEORIGIN`, solo se permite en el mismo dominio — el riesgo de clickjacking externo sigue bloqueado.

8.3 Estrategia en Producción (VPS)

```
# Nginx sirve media y static SIN pasar por Django
location /media/ {
    alias /var/www/ecppp/media/;
    add_header X-Content-Type-Options nosniff;
}
location /static/ {
    alias /var/www/ecppp/staticfiles/;
    expires 1y; # Cache agresivo – los static no cambian entre deploys
    add_header Cache-Control "public, immutable";
}
```

Beneficios de que Nginx sirva los archivos directamente:

1. **Performance:** Nginx es un servidor de archivos estáticos optimizado. No genera proceso Python para servir un PDF.

2. **Liberación de workers Unicorn:** Los workers Django quedan libres para procesar lógica de negocio.
3. **Cache HTTP:** Se pueden configurar headers de cache en Nginx sin tocar Django.

8.4 Escalabilidad Futura — Object Storage

Si el volumen de archivos supera la capacidad del disco del VPS, la migración natural es a un servicio de object storage (Amazon S3, DigitalOcean Spaces, MinIO self-hosted):

Actual: Django → MEDIA_ROOT (filesystem local)
Futuro: Django → django-storages → S3/Spaces bucket

Con `django-storages`, se cambia una línea en settings:

```
DEFAULT_FILE_STORAGE = "storages.backends.s3boto3.S3Boto3Storage"
```

El resto del código (modelos, vistas, templates) no cambia. Esta es la Arquitectura Hexagonal en práctica: el storage es un adaptador de infraestructura.

9. Despliegue

9.1 Justificación de la Estrategia VPS

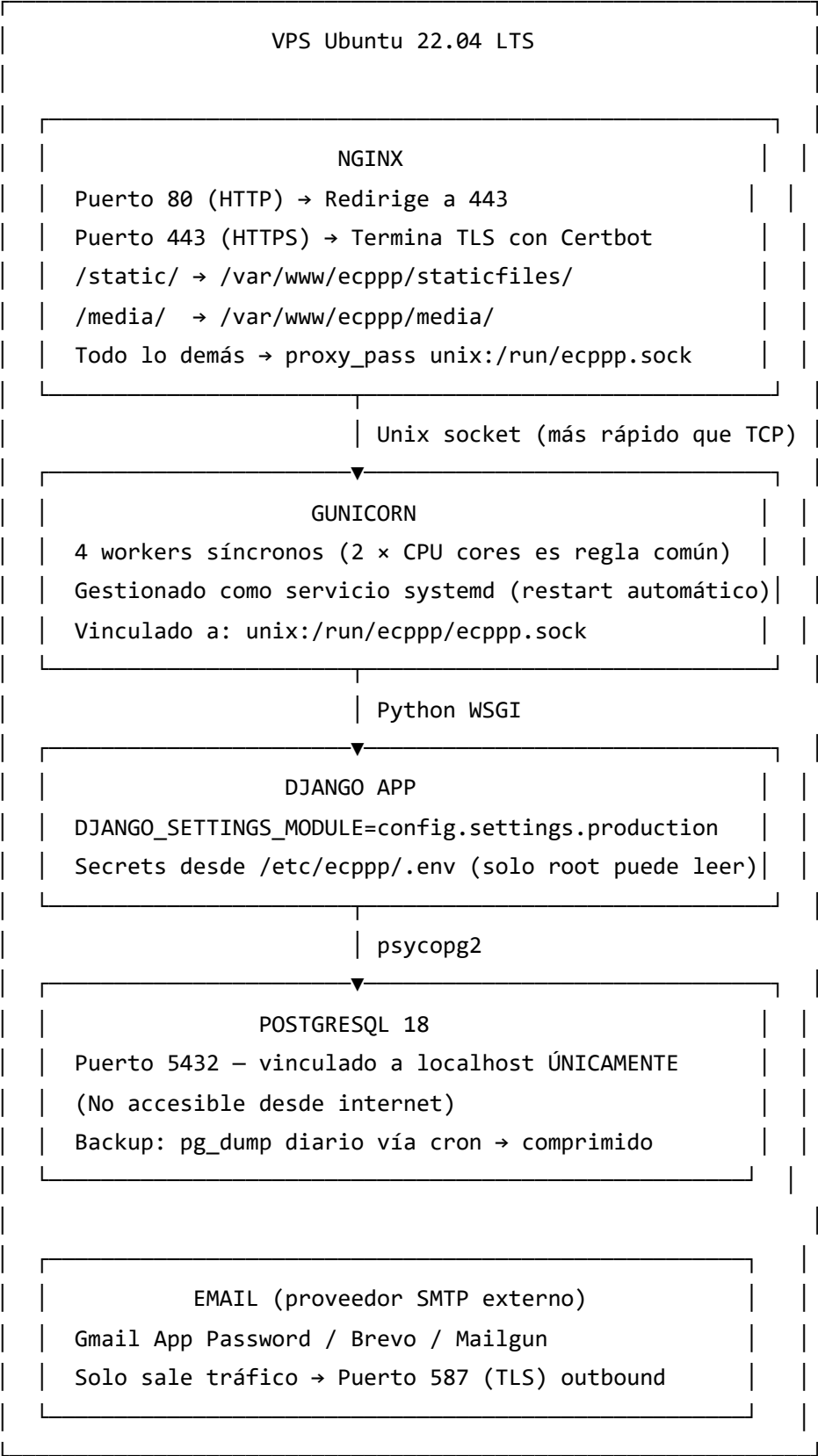
Se evaluaron tres opciones:

Opción	Ventajas	Desventajas	Decisión
PaaS (Heroku/Render/Railway)	Deploy automático, cero ops	Costo alto a escala, sin control de red, límites de almacenamiento de archivos	Descartado para producción
VPS propio	Control total, costo predecible, sin límites artificiales	Requiere configuración inicial	ELEGIDO

Opción	Ventajas	Desventajas	Decisión
Cloud gestionado (AWS/GCP)	Escala infinita, servicios gestionados	Complejidad operacional excesiva para el alcance del proyecto	Descartado en esta fase

Un VPS en DigitalOcean (Droplet 2GB RAM / 50GB SSD) cuesta ~12 USD/mes y es más que suficiente para el volumen de usuarios de una institución educativa.

9.2 Stack de Producción



| HTTPS :443

9.3 Por qué Nginx + Gunicorn (y no solo Django runserver)

`python manage.py runserver` es el servidor de desarrollo de Django. **No es apto para producción** por múltiples razones:

- Maneja una sola request a la vez (sin concurrencia).
- No sirve archivos estáticos en modo producción.
- No maneja certificados TLS.
- No tiene control de workers ni restart automático.

Gunicorn es el servidor WSGI de producción: implementa el protocolo WSGI (Web Server Gateway Interface) que Django expone, y maneja múltiples workers para concurrencia real.

Nginx actúa como reverse proxy delante de Gunicorn:

- Termina las conexiones TLS (Gunicorn no necesita saber de HTTPS).
- Sirve los archivos estáticos y media directamente sin consumir workers Django.
- Puede manejar miles de conexiones simultáneas con muy bajo overhead (event-driven, no threaded).
- Agrega rate limiting, logging y compresión gzip antes de que Django entre en escena.

9.4 Proceso de Deploy

```
1. git pull origin main           # Traer el código nuevo
2. source venv/bin/activate
3. pip install -r requirements.txt # Actualizar dependencias si cambió
4. python manage.py migrate       # Aplicar migraciones de BD
5. python manage.py collectstatic # Copiar static files a staticfiles/
6. systemctl restart ecppp       # Reiniciar Gunicorn (carga el nuevo código)
```

Nginx no necesita reiniciarse (solo sirve archivos, no corre código Python).

10. Calidad de Software — ISO 25010

La norma **ISO/IEC 25010:2011** define el modelo de calidad del producto software. El proyecto ECPPP adoptó esta norma como marco de referencia para las decisiones de diseño y las métricas de calidad.

10.1 Adecuación Funcional

Definición: El sistema proporciona funciones que satisfacen las necesidades declaradas e implícitas cuando se usa bajo condiciones especificadas.

Se divide en tres sub-características:

- **Completitud funcional:** ¿El sistema cubre todos los casos de uso requeridos?
- **Corrección funcional:** ¿Los resultados son correctos y precisos?
- **Pertinencia funcional:** ¿Las funciones implementadas son las apropiadas para el objetivo?

Evidencia en ECPPP:

Sub-característica	Evidencia concreta
Completitud	Gestión de usuarios con 4 roles, periodos, asignaturas, paralelos, matrículas, calificaciones, asistencia, solicitudes, notificaciones. Cada módulo con CRUD completo o en desarrollo.
Corrección	Validaciones de negocio implementadas con tests: cupo máximo de paralelos (50 estudiantes), duración de periodos (4-7 meses), horas lectivas máximas (60h), conflictos de horario docente/estudiante (HU21).
Pertinencia	Cada funcionalidad surge de historias de usuario documentadas en docs/ y especificadas en openspec/. No hay funcionalidad "por las dudas".

Pendiente en progreso: Los módulos de calificaciones, asistencia y solicitudes tienen sus entidades de dominio definidas y sus estructuras de BD creadas. La presentación (vistas) está en desarrollo activo.

10.2 Eficiencia de Desempeño

Definición: Relación entre el desempeño del sistema y los recursos utilizados bajo condiciones establecidas.

Sub-características: comportamiento temporal (tiempo de respuesta), utilización de recursos, capacidad.

Estrategias implementadas:

Técnica	Dónde	Impacto
Índices compuestos en BD	OTPToken , RegistroAuditoria	Búsquedas en $O(\log n)$ en vez de $O(n)$
Paginación en DRF	PAGE_SIZE = 20	Evita cargar 1000 registros en una respuesta
Nginx sirve static/media	Configuración de producción	Nginx es 10x más rápido que Django para archivos
select_related / prefetch_related	Consultas ORM	Evita el problema N+1 (N queries adicionales por cada ítem)
SESSION_COOKIE_AGE = 3600	Sesiones	Evita sesiones zombi que acumulan memoria

Pendiente: Redis para caché de consultas frecuentes (lista de docentes, asignaturas activas) cuando el sistema esté en producción con carga real.

10.3 Compatibilidad

Definición: Grado en que el sistema puede intercambiar información con otros sistemas o funcionar en el mismo entorno.

Evidencia en ECPPP:

- **API REST expuesta:** Django REST Framework permite a sistemas externos (otro portal, app móvil futura) consumir datos de ECPPP vía endpoints REST estándar con autenticación por sesión.

- **Internacionalización:** `LANGUAGE_CODE = "es-ec"` , `TIME_ZONE = "America/Guayaquil"` — el sistema habla el mismo idioma de fecha/hora que Ecuador.
- **Dominio desacoplado:** Las entidades de dominio son Python puro sin dependencias de Django. Pueden ser consumidas por cualquier capa, incluso un futuro servicio externo.
- **Email intercambiable:** El backend SMTP es configurable por variable de entorno (`EMAIL_BACKEND`). Se puede cambiar de Gmail a Brevo a Mailgun sin tocar el código.

10.4 Usabilidad

Definición: Grado en que el sistema puede ser usado con efectividad, eficiencia y satisfacción por usuarios específicos.

Sub-características: reconocibilidad, aprendizaje, operabilidad, protección contra errores del usuario, estética, accesibilidad.

Decisiones de diseño orientadas a usabilidad:

Decisión	Justificación
Sistema de diseño consistente con Tailwind	Mismos tokens de color, espaciado y tipografía en toda la app. El usuario no tiene que re-aprender el sistema visual en cada pantalla.
Mensajes de feedback con auto-dismiss (Alpine.js)	Los mensajes de éxito desaparecen solos a los 5 segundos. El usuario no necesita cerrarlos manualmente, pero puede hacerlo.
Tom Select para listas largas	Un <code><select></code> nativo con 50 docentes es inutilizable. Tom Select con búsqueda convierte ese problema en un campo de búsqueda.
Sidebar colapsable con persistencia	El inspector que trabaja en una pantalla pequeña puede colapsar el sidebar y la preferencia se recuerda entre sesiones (<code>localStorage</code>).
Validación server-side con errores en contexto	Los errores de formulario aparecen junto al campo que los causó, no en un mensaje genérico arriba.
Forzar cambio de contraseña en primer login	Mejora la seguridad sin requerir que el usuario la cambie "cuando se acuerde". El sistema lo guía activamente.

10.5 Fiabilidad

Definición: Grado en que el sistema ejecuta las funciones requeridas bajo condiciones especificadas durante un período de tiempo.

Sub-características: madurez, disponibilidad, tolerancia a fallos, recuperabilidad.

Mecanismos de fiabilidad:

Mecanismo	Qué previene
Bloqueo de cuenta tras 5 intentos	Ataques de fuerza bruta que degradan el servicio y comprometen cuentas
OTP de un solo uso con expiración	Tokens robados o reutilizados que comprometen el flujo de verificación
unique_together en BD	Datos duplicados que corrompieron el estado (ej.: dos asistencias el mismo día)
Transacciones atómicas	Estados inconsistentes si una operación multi-paso falla a mitad
systemd para Unicorn	Reinicio automático del servidor de aplicación si crashea
Umbral CI del 70%	Código sin cobertura no puede entrar a la rama principal
Migraciones en CI	Una migración rota no llega a producción

10.6 Seguridad

Definición: Grado en que el sistema protege información y datos contra accesos no autorizados.

Sub-características: confidencialidad, integridad, no repudio, responsabilidad, autenticidad.

(Ver Sección 7 para el detalle técnico completo de cada mecanismo.)

Sub-característica	Implementación en ECPPP
Confidencialidad	PBKDF2-SHA256 para contraseñas, HTTPS en producción, cookies HttpOnly

Sub-característica	Implementación en ECPPP
Integridad	CSRF en todos los formularios, validación de cédula única en BD
No repudio	<code>RegistroAuditoria</code> con IP y timestamp — no se puede negar una acción
Responsabilidad	Cada acción sensible está ligada a un usuario específico en el audit log
Autenticidad	Triple factor (email + password + rol) + OTP para verificación

10.7 Mantenibilidad

Definición: Grado en que el sistema puede ser modificado efectiva y eficientemente.

Sub-características: modularidad, reusabilidad, analizabilidad, modificabilidad, testeabilidad.

El aporte más grande a la mantenibilidad es la arquitectura hexagonal:

Si el módulo de email necesita cambiar de SMTP a un servicio de mensajería en tiempo real, solo se reemplaza `infrastructure/email_service.py` . El dominio y los casos de uso no saben que el transporte cambió.

Sub-característica	Evidencia
Modularidad	7 bounded contexts independientes. Modificar <code>calificaciones</code> no afecta <code>asistencia</code> .
Reusabilidad	<code>shared/</code> y <code>core/</code> centralizan utilitarios comunes. Los repositorios abstractos son reutilizables.
Analizabilidad	Código formateado con Black, lint con Flake8, tests con pytest — cualquier desarrollador puede analizar el estado del sistema.
Modificabilidad	La regla de dependencia unidireccional garantiza que cambiar infraestructura no rompe el dominio.
Testeabilidad	El dominio es Python puro — sus tests no necesitan BD ni Django. Son tests unitarios veloces.

10.8 Portabilidad

Definición: Grado en que el sistema puede ser transferido de un entorno a otro.

Sub-características: adaptabilidad, instalabilidad, reemplazabilidad.

Evidencia en ECPPT:

Sub-característica	Implementación
Adaptabilidad	Settings por entorno (<code>base / development / production</code>). Cambiar de entorno es cambiar <code>DJANGO_SETTINGS_MODULE</code> .
Instalabilidad	<code>requirements.txt</code> con versiones fijadas. <code>python -m venv + pip install</code> reproduce el entorno exacto en cualquier máquina.
Reemplazabilidad	Al seguir convenciones Django estándar, el proyecto puede ser tomado por cualquier developer Python sin curva de aprendizaje del framework.

Adicionalmente, la separación de configuración en variables de entorno sigue el principio **12-Factor App** (metodología para apps cloud-native), que es el estándar de la industria para portabilidad.

11. CI/CD y Testing

11.1 ¿Qué es CI/CD y por qué importa?

CI (Continuous Integration) significa que cada vez que un desarrollador sube código al repositorio, un sistema automatizado verifica que el código no rompe nada. En el proyecto de clase, esto garantiza que la calidad del código se mantiene sin depender de que alguien recuerde correr los tests manualmente.

CD (Continuous Delivery) es la extensión: automatizar también el deploy a producción después de que CI pasa. En ECPPT se tiene CI completo; el CD automático es el siguiente paso.

11.2 Pipeline GitHub Actions

Definido en `.github/workflows/ci.yml` . Se ejecuta automáticamente en cada `push` y en cada `pull request` :

GitHub Actions Runner

Step 1: flake8 .

→ ¿El código sigue el estilo PEP8?

→ Si falla: el pipeline se detiene. Nadie puede mergear

↓

Step 2: black --check .

→ ¿El código está formateado correctamente?

→ Si falla: el pipeline se detiene

↓

Step 3: python manage.py migrate

→ ¿Las migraciones son consistentes con el código?

→ Detecta si alguien olvidó correr makemigrations

↓

Step 4: coverage run -m pytest

→ ¿Pasan todos los tests?

→ ¿La cobertura está por encima del umbral?

↓

Step 5: coverage report --fail-under=70

→ Si la cobertura cayó del 70%: falla

→ Ningún PR se puede mergear sin cobertura suficiente

11.3 Estrategia de Testing

Se usan tres niveles de tests según la pirámide de testing:

- ▲ E2E (pocos, lentos)
- ▲▲▲ Integration (algunos)
- ▲▲▲▲▲ Unit (muchos, rápidos)

Unit Tests (tests unitarios):

- Prueban la lógica del dominio en Python puro, sin BD, sin Django.
- Veloces (milisegundos por test).
- Ejemplo: verificar que `SymbolValidator` rechaza una contraseña sin símbolos.

Integration Tests (tests de integración):

- Prueban el flujo completo: HTTP request → Vista → BD → Respuesta.

- Usan `pytest-django` con una BD PostgreSQL de test (se crea y destruye en cada ejecución).
- Ejemplo: hacer POST a `/usuarios/crear/` y verificar que el usuario existe en BD con el rol correcto.

Factory-boy para fixtures:

- En vez de crear datos de prueba a mano, `factory-boy` genera objetos válidos con datos aleatorios.
- `UsuarioFactory()` crea un usuario con `username`, `email`, `cédula` y `contraseña` válidos en una línea.
- Si el modelo cambia (se agrega un campo requerido), solo se actualiza la `Factory`.

TDD (Test-Driven Development):

Para las features más complejas (HU21 conflictos de horario, registro inmutable de usuarios), se siguió el ciclo estricto:

1. **RED:** Escribir el test que describe el comportamiento esperado. El test falla (el código no existe aún).
2. **GREEN:** Escribir el mínimo código necesario para que el test pase.
3. **REFACTOR:** Mejorar el código sin romper los tests.

Este ciclo garantiza que el test es real (efectivamente falla antes del código) y que el código implementado es exactamente lo que el test necesita.

12. Módulos del Sistema

12.1 Roles y Permisos

Rol	Descripción	Módulos accesibles
Secretaría	Administrador académico	Usuarios, Periodos, Asignaturas, Paralelos, Matrículas
Docente	Profesor asignado a paralelos	Calificaciones (sus paralelos), Asistencia (sus paralelos)
Inspector	Supervisor de asistencia	Asistencia (todos los paralelos), Solicitudes, Notificaciones

Rol	Descripción	Módulos accesibles
Estudiante	Alumno matriculado	Calificaciones propias, Asistencia propia, Solicitudes propias

12.2 Estado de Implementación

Completamente implementado:

- ✓ Autenticación triple (email + password + rol)
- ✓ OTP por email (verificación y reset de contraseña)
- ✓ Bloqueo de cuenta por intentos fallidos
- ✓ Middleware de contraseña temporal (primer login)
- ✓ CRUD de usuarios por Secretaría (cédula obligatoria e inmutable)
- ✓ Tipos de licencia y asignaturas
- ✓ Periodos académicos con validaciones (duración 4-7 meses)
- ✓ Paralelos con cupo máximo, bloque horario
- ✓ Detección de conflictos de horario (docente y estudiante) — HU21
- ✓ Matrículas
- ✓ Dashboard diferenciado por rol
- ✓ Sidebar colapsable con persistencia
- ✓ Notificaciones internas

En progreso:

- ☐ Vistas de calificaciones (dominio y BD definidos)
- ☐ Vistas de asistencia (dominio y BD definidos)
- ☐ Vistas de solicitudes (dominio y BD definidos)

13. Banco de Preguntas y Respuestas

BLOQUE A — Arquitectura

P1. ¿Qué arquitectura usa el proyecto y por qué se eligió?

Arquitectura Hexagonal (Ports & Adapters) con DDD. Se eligió para aislar la lógica de negocio de Django, permitiendo cambiar el ORM, el motor de BD o el framework de presentación sin tocar el dominio.

P2. ¿Cómo se separa la lógica de negocio de la infraestructura?

Cada app tiene una capa `domain/` en Python puro (sin imports de Django). La capa `infrastructure/` contiene los modelos ORM y es la única que "toca" la base de datos. La regla es: si un archivo en `domain/` tiene `import django`, hay un error arquitectónico.

P3. ¿Qué es un Bounded Context en este proyecto?

Es una app (`usuarios`, `academico`, `calificaciones`, etc.) con su propio modelo de dominio, reglas de negocio y repositorios. Cada contexto es independiente; el término "Paralelo" dentro de `academico` tiene un significado preciso que no se mezcla con otros contextos.

P4. ¿Cómo se comunican los bounded contexts entre sí?

A través de FKs de base de datos en la infraestructura y referencias por ID en el dominio. No hay imports cruzados entre capas de dominio; la integración ocurre en la capa de aplicación o infraestructura.

P5. ¿Por qué no usaron microservicios?

El tamaño del equipo y el alcance del proyecto hacen que un monolito modular sea la decisión correcta. La arquitectura hexagonal permite migrar a microservicios extrayendo bounded contexts si fuera necesario; las interfaces ya están definidas.

P6. ¿Qué es un Value Object y dónde lo usan?

Es un objeto inmutable definido por sus atributos, sin identidad propia. En el proyecto se implementan como `@dataclass(frozen=True)` — por ejemplo, `UsuarioEntity`, `PeriodoEntity`. No tienen ID porque no representan "el mismo objeto" — representan un estado de datos.

P7. ¿Qué rol cumple el Repository Pattern?

Define interfaces en `domain/repositories.py` (qué se puede hacer con los datos) y las implementaciones en `infrastructure/repositories.py` (cómo se hace con Django ORM). El dominio nunca sabe que existe PostgreSQL; si se cambia el motor, solo se reemplaza la implementación.

P8. ¿Qué son los "Puertos" y "Adaptadores" en su arquitectura?

Los Puertos son las interfaces abstractas que el dominio define (`IUsuarioRepository`). Los Adaptadores son las implementaciones concretas que conectan esas interfaces con el mundo exterior (`DjangoUsuarioRepository` que usa ORM, o `BrevoEmailBackend` que usa la API de Brevo).

BLOQUE B — Tecnología y Librerías

P9. ¿Qué librerías usan para estilizar la interfaz?

Tailwind CSS 3.4.17 como framework CSS de utilidades, Alpine.js 3.15.9 para reactividad ligera en el HTML, y Tom Select 2.3.1 para selects enriquecidos con búsqueda. La tipografía es Inter (Google Fonts). Todos cargados desde CDN actualmente.

P10. ¿Por qué eligieron Tailwind y no Bootstrap?

Tailwind permite un sistema de diseño más preciso con tokens configurables (colores de marca, sombras, tipografía). Evita el CSS sobrante que genera Bootstrap y no impone componentes visuales — se construye cada componente desde cero con clases utilitarias.

P11. ¿Qué hace Alpine.js en el proyecto?

Maneja estado reactivo pequeño directamente en el HTML: sidebar colapsable con `x-data`, modales de confirmación, auto-dismiss de alertas con `x-init="setTimeout()"`, toggle de contraseñas. Sin Alpine.js esto requeriría escribir JavaScript imperativo para cada interacción.

P12. ¿Por qué no usaron React o Vue para el frontend?

No se justifica la complejidad de un SPA para este dominio. Django Templates con SSR tiene renderizado más rápido en first load, no requiere un proceso de build separado, y Alpine.js cubre todas las interacciones reactivas necesarias. El deploy es más simple.

P13. ¿Por qué Django y no FastAPI o Flask?

Django incluye ORM, migraciones, autenticación, admin, CSRF y sesiones listos para usar. Para un proyecto con modelo de datos complejo y múltiples entidades relacionadas, Django reduce el tiempo de desarrollo y garantiza convenciones establecidas. Flask o FastAPI requerirían integrar cada pieza por separado.

P14. ¿Usan Django REST Framework? ¿Para qué?

Sí, DRF 3.16.0 expone endpoints REST en el módulo académico. Actualmente la autenticación es por sesión (compatible con el login web). DRF habilita la integración futura con apps móviles u otros sistemas externos sin reescribir la lógica.

P15. ¿Cómo manejan las variables de entorno?

Con `python-dotenv` : las variables se definen en `.env` local (en `.gitignore`) y se cargan en `settings/base.py` . El `.env.example` documenta todas las variables requeridas. Ningún secreto jamás entra al repositorio.

BLOQUE C — Base de Datos

P16. ¿Por qué eligieron SQL (PostgreSQL) y no NoSQL?

Los datos son altamente relacionales: usuarios → matrículas → paralelos → calificaciones. PostgreSQL garantiza integridad referencial (FKs con restricciones), ACID y consistencia que NoSQL no provee por defecto. MongoDB agregaría complejidad sin beneficio real dado que no hay datos semiestructurados.

P17. ¿Cómo funciona la base de datos actualmente?

PostgreSQL 18 con el ORM de Django. Cada app tiene sus modelos con FKs entre ellos. Las migraciones versionan el esquema en git. En desarrollo se conecta a una instancia local; en producción, a una instancia en el VPS accesible solo por localhost.

P18. ¿Tienen índices en la base de datos?

Sí. Índices compuestos en `OTPToken` (`usuario`, `usado`, `expira_en`) para buscar tokens vigentes, y en `RegistroAuditoria` (`usuario`, `accion`, `timestamp`) para buscar el historial de un usuario. Reducen la búsqueda de $O(n)$ a $O(\log n)$.

P19. ¿Cómo manejan las migraciones de base de datos?

Con `makemigrations` / `migrate` de Django. Las migraciones son código Python versionado en git junto al código fuente. El CI/CD corre `migrate` automáticamente para verificar que las migraciones son consistentes con los modelos.

P20. ¿El modelo Usuario usa el sistema de auth nativo de Django?

Sí, `Usuario` extiende `AbstractUser` via `Template Method`. Heredamos gratis: hashing de contraseñas, permisos, grupos, sesiones y Django Admin. Solo agregamos rol, cédula, teléfono y

los campos de seguridad específicos del dominio.

P21. ¿Por qué `RegistroAuditoria` usa `SET_NULL` en vez de `CASCADE` ?

Porque si se elimina un usuario del sistema, sus registros de auditoría deben mantenerse por trazabilidad y evidencia. Con `CASCADE` , borrar un usuario borraría todas sus acciones registradas — potencialmente evidencia de actividad maliciosa o errores.

BLOQUE D — Seguridad

P22. ¿Están usando algún método de encriptación para contraseñas?

Sí. Django aplica PBKDF2 con SHA-256 y salt aleatorio automáticamente. Las contraseñas NUNCA se almacenan en texto plano. El formato almacenado es `pbkdf2_sha256$870000$salt$hash` . Ningún campo de la BD contiene la contraseña original.

P23. ¿Qué es PBKDF2 y por qué es seguro?

Password-Based Key Derivation Function 2: aplica SHA-256 con un salt aleatorio 870.000 iteraciones. El salt hace que dos usuarios con la misma contraseña tengan hashes distintos (elimina rainbow tables). Las iteraciones hacen que descifrar una contraseña robada tome meses con hardware moderno.

P24. ¿Cómo previenen ataques de fuerza bruta en el login?

Tras 5 intentos fallidos, la cuenta se bloquea por 15 minutos. Los intentos y la fecha de desbloqueo se persisten en el modelo `Usuario` . El límite y el tiempo son configurables en `settings.py` como `MAX_LOGIN_ATTEMPTS` y `ACCOUNT_LOCKOUT_MINUTES` .

P25. ¿Qué es el OTP y para qué se usa?

One-Time Password: código numérico de 6 dígitos enviado por email con expiración de 10 minutos. Se usa para verificación de identidad en flujos sensibles (primer login, reset de contraseña). El token se marca `usado=True` tras su uso, evitando reutilización incluso si no expiró.

P26. ¿Cómo protegen contra CSRF?

`CsrfViewMiddleware` está activo en todos los formularios con `{% csrf_token %}` . El token es único por sesión y se verifica en cada POST. En producción, `CSRF_COOKIE_SECURE = True` garantiza que la cookie solo viaja por HTTPS, eliminando la posibilidad de interceptación.

P27. ¿Qué hace el middleware `ForzarCambioPasswordMiddleware` ?

Intercepta cada request de un usuario autenticado y verifica si `debe_cambiar_password = True` . Si es así, redirige obligatoriamente al formulario de cambio de contraseña sin importar la URL destino. Se activa cuando Secretaría crea un usuario con contraseña temporal.

P28. ¿Cómo manejan los secretos del sistema?

Todas las credenciales (`SECRET_KEY`, `DATABASE_PASSWORD`, `EMAIL_HOST_PASSWORD`) se cargan desde variables de entorno. El `.env` está en `.gitignore` . En producción, el archivo de secrets tiene permisos de solo lectura para el usuario del sistema.

BLOQUE E — Almacenamiento de Archivos

P29. ¿Cómo manejan el almacenamiento de archivos como PDFs e imágenes?

Django almacena los archivos subidos en el filesystem local bajo `media/` (configurado en `MEDIA_ROOT`). En producción, Nginx sirve esos archivos directamente (con `alias`) sin pasar por Django para máximo rendimiento.

P30. ¿Por qué se permite `X-Frame-Options: SAMEORIGIN` y no `DENY` ?

El módulo del inspector necesita previsualizar PDFs dentro de `<iframe>` en la misma página. Con `DENY` , los iframes internos se bloquearían, rompiendo esa funcionalidad. `SAMEORIGIN` solo permite embeber contenido del mismo dominio — el riesgo de clickjacking externo sigue bloqueado.

P31. ¿Qué sucede si los archivos crecen mucho en el VPS?

La migración natural es `django-storages` con un bucket S3 o DigitalOcean Spaces. Al ser la infraestructura un adaptador intercambiable, el cambio es una línea en settings (`DEFAULT_FILE_STORAGE = S3Boto3Storage`). El dominio, los casos de uso y las vistas no se tocan.

P32. ¿Cómo se protege el acceso a archivos privados?

Las URLs de media son directas actualmente. Para archivos con requisitos de privacidad se implementará validación de permisos en el view que genera la URL, o signed URLs temporales (similar a las S3 presigned URLs) que expiran después de X segundos.

BLOQUE F — Despliegue

P33. ¿Cómo se realizará el despliegue del proyecto?

En un VPS Ubuntu 22.04 con Nginx como reverse proxy (termina TLS, sirve static/media), Gunicorn como servidor WSGI de Django (4 workers), PostgreSQL como BD (accesible solo por localhost), y Certbot para el certificado TLS de Let's Encrypt.

P34. ¿Por qué un VPS y no Heroku, Railway o Render?

Control total sobre configuración de red, sin límites artificiales de RAM o almacenamiento, costo predecible (~12 USD/mes), y acceso root para configurar PostgreSQL y Nginx según las necesidades. Para un sistema académico institucional, la autonomía operacional es prioritaria.

P35. ¿Por qué Gunicorn y no `python manage.py runserver` ?

`runserver` es el servidor de desarrollo: maneja una sola request simultánea, no tiene control de workers y no está diseñado para producción. Gunicorn implementa WSGI con múltiples workers concurrentes, restart automático via systemd y es el estándar de la industria para Django en producción.

P36. ¿Cuál es el rol específico de Nginx?

Nginx actúa como "guardián" frente a Gunicorn: termina las conexiones HTTPS (Gunicorn no necesita manejar TLS), sirve los archivos static y media directamente sin consumir un worker Django, maneja miles de conexiones con mínimo overhead, y permite rate limiting y logging centralizado.

P37. ¿Cómo se manejan las actualizaciones en producción?

`git pull` → `pip install` (si cambió requirements) → `migrate` → `collectstatic` → `systemctl restart ecppp` . Nginx no necesita reiniciarse. Las migraciones de Django son seguras porque el ORM verifica el estado actual de la BD antes de aplicar cambios.

P38. ¿Tienen plan de rollback?

Las migraciones Django son reversibles con `migrate <app> <numero_anterior>` . El código se puede revertir con `git checkout <tag_anterior>` . Los snapshots del VPS (ofrecidos por todos los proveedores) permiten restaurar el servidor completo ante un fallo catastrófico.

BLOQUE G — Testing y Calidad

P39. ¿Qué herramientas de testing usan?

`pytest` como framework, `pytest-django` para fixtures de Django (base de datos de test, cliente HTTP), `factory-boy` para generación de objetos de test, y `coverage` para medir cobertura. El umbral mínimo forzado en CI es 70%.

P40. ¿Por qué 70% de cobertura y no 100%?

El 100% tiene rendimientos decrecientes: cubrir el último 30% normalmente cubre código trivial (getters, **str**) a costo de tiempo de desarrollo desproporcionado. El 70% garantiza cobertura de la lógica crítica (auth, validaciones de negocio, reglas académicas) que es donde los bugs más dañinos viven.

P41. ¿Qué es TDD y lo aplican en el proyecto?

Test-Driven Development: escribir el test ANTES que el código (RED → GREEN → REFACTOR). Se aplica en Strict Mode para las features más complejas (HU21 conflictos de horario, registro inmutable). Garantiza que los tests son reales (efectivamente fallan antes del código) y el código hace exactamente lo que el test necesita.

P42. ¿Cómo funciona el pipeline de CI?

GitHub Actions ejecuta automáticamente en cada push/PR: `flake8` → `black` → `migrate` → `pytest` con `coverage` → umbral 70%. Si cualquier paso falla, el PR no puede mergearse. Nadie puede "olvidarse" de correr los tests.

P43. ¿Qué es `flake8` y `black` y por qué los usan juntos?

`Flake8` verifica conformidad con PEP8 (estilo oficial de Python): líneas largas, imports no usados, espaciado. `Black` formatea el código automáticamente de forma uniforme y "opinada". Juntos eliminan los debates de estilo en code review y garantizan consistencia total en el codebase.

BLOQUE H — Patrones de Diseño

P44. ¿Qué patrones de diseño utilizan y por qué?

Repository (desacopla dominio de BD), Strategy (backends de auth intercambiables), Template Method (AbstractUser personalizable), Decorator (middleware y `@login_required`), Observer

(Django signals para eventos), Factory (factory-boy en tests), Chain of Responsibility (middleware stack).

P45. ¿Cómo aplican el patrón Strategy en la autenticación?

`AUTHENTICATION_BACKENDS` define una lista de estrategias. Django prueba cada una en orden. `ECPPPAuthBackend` autentica por email+password+rol. `ModelBackend` es el fallback para el admin Django. Agregar OAuth sería agregar una tercera estrategia sin tocar las existentes.

P46. ¿Cómo aplican SOLID en el proyecto?

S: cada capa tiene una responsabilidad (domain=reglas, infra=BD, presentation=HTTP). O: los repositorios son interfaces extensibles sin modificar. L: `ECPPPAuthBackend` sustituye a `ModelBackend` correctamente. I: cada repositorio expone solo lo que su contexto necesita. D: los casos de uso dependen de interfaces, no de Django ORM directamente.

BLOQUE I — Proceso y Metodología

P47. ¿Qué metodología de desarrollo usan?

Desarrollo iterativo por sprints con SDD (Spec-Driven Development) para features complejas. Cada cambio significativo pasa por: exploración → propuesta → especificación → diseño técnico → tareas → implementación → verificación. La documentación de cada fase queda en `openspec/`.

P48. ¿Cómo gestionan las ramas en git?

Feature branches desde `develop`: `feature/<nombre>` para nuevas funcionalidades, `hotfix/<nombre>` para correcciones urgentes. Todo se integra a `develop` via Pull Request con CI aprobado. No se mergea sin que todos los checks pasen.

P49. ¿Qué es SDD?

Spec-Driven Development: documenta el cambio con especificaciones y diseño técnico antes de escribir código. El equipo acuerda el "qué" y el "por qué" antes del "cómo", evitando reescrituras por malentendidos. Los artefactos (spec, diseño, tareas) quedan en `openspec/` como documentación viva.

P50. ¿Cómo planifican las funcionalidades que faltan?

Los módulos pendientes (calificaciones, asistencia, solicitudes completos) tienen sus entidades de dominio definidas y su esquema de BD creado. El backlog se prioriza por dependencias: primero se termina lo que otros módulos necesitan.

BLOQUE J — Preguntas Adicionales

P51. ¿El sistema maneja internacionalización?

Sí, `LANGUAGE_CODE = "es-es"` y `TIME_ZONE = "America/Guayaquil"`. Los mensajes de validación están en español. El sistema de `i18n` de Django (`gettext`) está disponible para agregar más idiomas si la institución lo necesitara.

P52. ¿Cómo manejan errores 404 y 500 en producción?

Con `DEBUG=False`, Django renderiza páginas de error personalizadas (`templates/404.html`, `templates/500.html`). Los errores 500 pueden configurarse para enviarse por email al administrador vía `ADMINS + logging` de Django.

P53. ¿Qué diferencia hay entre `staticfiles` y `media` ?

`staticfiles` son archivos del código (CSS, JS, imágenes del sistema) — versionados con el código, se copian con `collectstatic` en cada deploy. `media` son archivos subidos por usuarios en tiempo de ejecución — dinámicos, viven fuera del código, se sirven por Nginx en producción.

P54. ¿El sistema envía emails? ¿Cómo está configurado?

Sí vía SMTP. Los emails cubren: OTP de verificación, reset de contraseña y notificaciones. El backend es configurable por variable de entorno: Gmail, Brevo o Mailgun sin cambiar el código. Para Brevo (necesario en Ecuador por restricciones de hostname), hay un backend custom `BrevoEmailBackend`.

P55. ¿Cómo escala el sistema si crece el número de usuarios?

Corto plazo: más workers Unicorn, índices adicionales. Mediano plazo: Redis para caché de sesiones y queries frecuentes. Largo plazo: separar PostgreSQL a un servidor dedicado o servicio gestionado (RDS, DigitalOcean Managed DB). La arquitectura hexagonal facilita estos cambios sin tocar el dominio.

P56. ¿Cómo garantizan que la cédula sea única e inmutable?

`unique=True` en el campo `cedula` — garantía a nivel de base de datos (UNIQUE constraint). La inmutabilidad se enforces en la capa de presentación: el campo `cedula` se excluye del formulario de edición y no existe botón de edición en la UI post-creación. La corrección se hace mediante `delete + recreate`.

P57. ¿Tienen panel de administración?

Sí, Django Admin en `/admin/`. Accesible solo a superusuarios con `is_staff=True`. Se usa para gestión de datos internos, soporte técnico y correcciones directas. No es parte del flujo normal de usuarios del sistema.

P58. ¿Por qué configuran `X_FRAME_OPTIONS = "SAMEORIGIN"` y no el más seguro `DENY` ?

El módulo del inspector requiere previsualizar PDFs de evidencia dentro de `<iframe>` en la misma página del sistema. Con `DENY`, ese `iframe` sería bloqueado por el browser. `SAMEORIGIN` permite iframes solo desde el mismo dominio, bloqueando clickjacking desde dominios externos, que es el vector real de ataque.

P59. ¿Qué es el `AUTH_USER_MODEL` personalizado y por qué es importante configurarlo desde el inicio?

Le dice a Django qué modelo usar como "usuario del sistema". Debe definirse antes de la primera migración: si se cambia después, las tablas de auth de Django están creadas con el modelo equivocado y la migración se vuelve muy compleja. Al extender `AbstractUser` desde el comienzo, se heredan todos los mecanismos de auth de Django más los campos propios del dominio.

P60. ¿Qué ventaja tiene que el dominio sea Python puro (sin Django)?

Los tests del dominio son tests unitarios veloces (milisegundos) que no necesitan BD ni servidor. Se pueden correr miles de veces durante el desarrollo. Además, si Django se deprecara o se decidiera migrar a FastAPI, la lógica de negocio (el activo más valioso del sistema) no necesita reescribirse.